

Diff-GNN-Order for Scheduling-Aware Hardware/Software Partitioning

Anonymous Authors

April 2026

Abstract

Hardware/software (HW/SW) partitioning is ultimately a schedule-sensitive problem: the best partition is not only the one with low isolated execution cost, but the one that interacts favorably with precedence constraints, software serialization, hardware concurrency, communication, and hardware-area limits. This paper focuses on the order-aware differentiable graph-neural implementation in our repository, namely **Diff-GNN-Order** (`diff_gnn_order`). The proposed method learns relaxed hardware assignments and software execution priorities jointly, optimizes a differentiable makespan surrogate, and then decodes the relaxed solution through feasibility repair and hybrid list-scheduling post-processing. The placement-only **Diff-GNN** variant is treated as an ablation of the proposed model.

The experimental protocol is defined directly from the repository artifacts. Real benchmarks are TOSA task graphs stored under the `inputs` folder, with SqueezeNet-TOSA as the primary benchmark and additional PyTorch- and TFLite-derived graphs used for cross-graph evaluation. Each concrete task-graph instance is created by combining a fixed topology with sampled execution, area, and communication costs parameterized by hardware scale, variance, communication scale, and area budget. We evaluate partitions using DAG makespan, static-priority makespan, and learned-priority makespan; for non-order-aware baselines we report the static-priority result, while for **Diff-GNN-Order** we report the better of the learned and static schedules. To study scalability, we further introduce SqueezeNet-like synthetic DAGs with 1,000 and 10,000 nodes whose edge budget preserves the edge-to-node ratio of the original SqueezeNet-TOSA graph. The resulting manuscript provides a paper-ready experimental-settings and results template aligned with the ICCAD notation and the current implementation.

1 Introduction

Hardware/software co-design requires a partition of a task graph into software-executed and hardware-accelerated tasks such that end-to-end latency is minimized without violating the available hardware area.

Although the partition is often represented as a binary labeling over nodes, the final quality of that labeling is determined by scheduling. Moving a task to hardware may shorten its local execution time, but it may also introduce communication overhead, alter the critical path, or change the serialization pattern on the software processor. This tight coupling between partitioning and scheduling is one of the central reasons the HW/SW partitioning problem remains difficult [1, 2, 3].

Classical exact methods are useful on small graphs, but their scalability is limited when realistic precedence and resource constraints are modeled together [1, 2]. Metaheuristics improve scalability by searching the binary placement space through repeated black-box evaluations, but they still depend on many expensive schedule evaluations and may struggle when the objective landscape is highly nonconvex. Graph-based learning offers a more structural alternative. In particular, Zheng *et al.* introduced a graph-convolution-based HW/SW partitioner that uses graph features and scheduling feedback to improve partition quality [4]. The current repository follows that direction and extends it toward a differentiable, order-aware formulation.

This paper is centered on **Diff-GNN-Order**, the order-aware implementation exposed in the code as `diff_gnn_order`. The method predicts both relaxed hardware-assignment probabilities and software execution priorities. These outputs are fed into a differentiable scheduler surrogate that approximates makespan under precedence, communication, and single-processor software serialization. During inference, the relaxed solution is decoded, repaired to satisfy the area constraint, optionally refined with local search, and evaluated with both learned and static software priorities. In contrast, the placement-only **Diff-GNN** model does not predict an execution order and is therefore treated here as an ablation rather than as the main method.

Our contributions are as follows:

- We formulate the proposed method as an order-aware differentiable graph-neural model for HW/SW partitioning, using the same notation and scheduling-oriented objective structure as the ICCAD draft.
- We document the concrete **Diff-GNN-Order** configuration used in the repository, including the feature

construction, edge-weight mechanism, ordering relaxation, and post-processing pipeline used during evaluation.

- We define an evaluation protocol that separates DAG makespan, static-priority makespan, and learned-priority makespan, and we make the reporting policy explicit: static-priority makespan for all non-order-aware methods, and $\min(M_{\text{static}}, M_{\text{learned}})$ for **Diff-GNN-Order**.
- We describe the repository datasets in a paper-ready format, including TOSA-derived real graphs, the cached SqueezeNet-TOSA instance sweep, and a new SqueezeNet-like synthetic generator for 1,000- and 10,000-node stress tests.

The remainder of the paper is organized as follows. Section 2 formulates the problem and its differentiable relaxation. Section 3 reviews prior exact, metaheuristic, and graph-based approaches. Section 4 presents **Diff-GNN-Order** and clarifies how **Diff-GNN** is used as an ablation. Section 5 details the experimental settings, Section 6 provides the results templates and reporting structure, and Section 7 concludes.

2 Problem Setup

We consider a target architecture with one software processor and a hardware substrate with limited total area. Software tasks execute sequentially on the CPU, while hardware tasks can overlap subject to precedence constraints. Communication between tasks assigned to different platforms incurs an edge-dependent delay. The goal of HW/SW partitioning is therefore to determine both a placement and an implied execution order that minimize end-to-end latency under the hardware-area budget.

2.1 Discrete Problem Formulation

Let $\mathcal{G}(\mathcal{V}, \mathcal{E})$ be a directed acyclic task graph (DAG). Each node $i \in \mathcal{V}$ has software execution cost t_i^{sw} , hardware execution cost t_i^{hw} , and hardware area cost A_i . Each edge $(j, i) \in \mathcal{E}$ encodes both a precedence relation and a communication cost C_{ji} that is paid when tasks j and i are assigned to different platforms.

We represent a discrete partition by a binary vector $\mathbf{p} \in \{0, 1\}^{|\mathcal{V}|}$, where $p_i = 1$ means that task i is mapped to hardware and $p_i = 0$ means that task i remains in software. Since software tasks share one processor, the final makespan also depends on the software order π^{sw} . Given $(\mathbf{p}, \pi^{sw})$, let $(\mathbf{s}, \mathbf{f}) = \text{Sched}(\mathbf{p}, \pi^{sw})$ denote the induced

start-time and finish-time vectors. The true objective is

$$\min_{\mathbf{p} \in \{0, 1\}^{|\mathcal{V}|}, \pi^{sw}} M(\mathbf{p}, \pi^{sw}) = \max_{i \in \mathcal{V}} f_i \quad (1)$$

$$\text{s.t.} \quad \sum_{i \in \mathcal{V}} p_i A_i \leq A_{\max}, \quad (2)$$

$$f_i = s_i + p_i t_i^{hw} + (1 - p_i) t_i^{sw}, \quad \forall i \in \mathcal{V}, \quad (3)$$

$$s_i \geq f_j + \mathbb{I}[p_j \neq p_i] C_{ji}, \quad \forall (j, i) \in \mathcal{E}, \quad (4)$$

$$s_j \geq f_i, \quad \forall i, j \in \mathcal{V} : p_i = p_j = 0 \text{ and } i \prec_{\pi^{sw}} j, \quad (5)$$

$$s_i \geq 0, \quad \forall i \in \mathcal{V}. \quad (6)$$

Equation (4) enforces precedence and cross-platform communication delay, while Equation (5) enforces single-processor serialization for software tasks.

2.2 Differentiable Relaxation

Directly optimizing Equations (1)–(6) is difficult because both placement and ordering are discrete. In **Diff-GNN-Order**, the binary placement vector is relaxed to $\tilde{\mathbf{p}} \in [0, 1]^{|\mathcal{V}|}$, where $\tilde{p}_i$ is the confidence that node i should be implemented in hardware. The software order is relaxed through a continuous software-priority vector $\tilde{\pi}^{sw} \in \mathbb{R}^{|\mathcal{V}|}$.

Under this relaxation, the execution term becomes

$$\tilde{t}_i(\tilde{\mathbf{p}}) = \tilde{p}_i t_i^{hw} + (1 - \tilde{p}_i) t_i^{sw}, \quad (7)$$

and the communication term becomes

$$\tilde{c}_{ji}(\tilde{\mathbf{p}}) = |\tilde{p}_j - \tilde{p}_i| C_{ji}. \quad (8)$$

The discrete scheduler is replaced by a differentiable surrogate

$$(\tilde{\mathbf{s}}, \tilde{\mathbf{f}}) = \widetilde{\text{Sched}}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}), \quad (9)$$

which approximates both DAG readiness and software serialization.

Since the max operator is nondifferentiable, the surrogate uses a smooth maximum

$$\text{SMax}_\beta(\{a_k\}) = \frac{1}{\beta} \log \sum_k e^{\beta a_k}, \quad (10)$$

where $\beta > 0$ controls approximation sharpness. Let $\text{Pred}(i)$ be the predecessor set of node i . The differentiable DAG-readiness term is

$$s_i^{dag} = \begin{cases} 0, & \text{Pred}(i) = \emptyset, \\ \text{SMax}_\beta(\{\tilde{f}_j + \tilde{c}_{ji}(\tilde{\mathbf{p}}) \mid j \in \text{Pred}(i)\}), & \text{otherwise.} \end{cases} \quad (11)$$

The software-readiness term is

$$s_i^{sw} = \text{SMax}_\beta(\{\tilde{f}_j + \phi_{sw}(\tilde{\pi}_j^{sw}, \tilde{\pi}_i^{sw}) \mid j \neq i\}), \quad (12)$$

where $\phi_{sw}(\cdot)$ is a differentiable ordering function derived from the learned software priorities.

The relaxed start and finish times are then

$$\tilde{s}_i = \text{SMax}_\beta(\{s_i^{dag}, s_i^{sw}\}), \quad (13)$$

$$\tilde{f}_i = \tilde{s}_i + \tilde{t}_i(\tilde{\mathbf{p}}), \quad (14)$$

and the differentiable makespan surrogate is

$$\widetilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) = \text{SMax}_\beta(\{\tilde{f}_i \mid i \in \text{Sink}(\mathcal{G})\}). \quad (15)$$

The relaxed area penalty is

$$\mathcal{L}_{\text{area}}(\tilde{\mathbf{p}}) = \left[\max \left(0, \sum_{i \in \mathcal{V}} \tilde{p}_i A_i - A_{\max} \right) \right]^2. \quad (16)$$

The proposed method minimizes the differentiable makespan surrogate together with the area penalty and additional regularization terms described in Section 4.

3 Related Work

3.1 Exact and Metaheuristic HW/SW Partitioning

HW/SW partitioning has traditionally been studied as a constrained combinatorial optimization problem. Surveys by Arató *et al.*, Hou *et al.*, and Teich describe a broad range of exact approaches, including dynamic programming, integer programming, and branch-and-bound, together with heuristic and metaheuristic alternatives [1, 2, 3]. Exact optimization remains valuable for small instances, but runtime degrades rapidly as the DAG becomes larger and the evaluation function becomes more scheduling-aware. This is precisely why population-based methods such as PSO, binary PSO, CLPSO, cooperative coevolutionary PSO, differential-evolution variants, genetic algorithms, and simulated annealing remain attractive baselines [5, 6, 7, 8, 9, 10, 11, 12].

3.2 Graph-Based Partitioning

Graph neural networks are a natural fit for HW/SW partitioning because the input is already a graph and the output is a node-level assignment. Zheng *et al.* proposed a graph-convolution-based HW/SW partitioner that combines graph features, graph convolution, and schedule feedback to improve solution quality on large task graphs [4]. Their method motivates the present work in two ways. First, graph features such as hardware gain and gain per unit area are directly useful for partitioning. Second, the schedule should not be treated as a purely downstream concern. The repository implementation studied in this paper builds on both observations and pushes the second one further by explicitly learning a software execution order inside the optimization loop.

3.3 Differentiable Ordering

The key methodological difference in our work is the use of a differentiable ordering head together with Sinkhorn-based soft permutations and a differentiable resource-order surrogate. These tools connect the method to differentiable ranking and latent-permutation models [13]. In the current paper, however, their purpose is not generic ranking; rather, they are used to approximate the single-processor software order induced by a candidate HW/SW partition. This schedule-aware emphasis is what distinguishes **Diff-GNN-Order** from placement-only graph partitioners and from population-based black-box baselines.

4 Proposed

Diff-GNN-Order

Method:

This paper treats **Diff-GNN-Order** as the proposed method. In the codebase, it corresponds to the `diff_gnn_order` implementation. The placement-only **Diff-GNN** model is a reduced variant that removes the ordering heads and is considered only in ablation studies.

4.1 Input Feature Preparation

Node features. For each node $i \in \mathcal{V}$, the raw node feature vector is constructed from execution, area, and graph-structural information:

$$\mathbf{f}_i^{\text{raw}} = \left[t_i^{sw}, t_i^{hw}, A_i, d_i, \frac{t_i^{hw}}{t_i^{sw} + \epsilon}, t_i^{sw} - t_i^{hw}, \frac{t_i^{sw} - t_i^{hw}}{\max(A_i, \epsilon)} \right]. \quad (17)$$

The first five terms are the default execution and resource descriptors; the last two correspond to the hardware-gain features inspired by **GCPS** [4]. In the repository, the strongest order-aware configuration uses the hybrid `default_plus_paper` profile, which concatenates both descriptor sets after normalization.

Edge features. For each edge $(j, i) \in \mathcal{E}$, the raw edge feature vector is

$$\mathbf{e}_{ji}^{\text{raw}} = [C_{ji}, |g_j - g_i|, |ga_j - ga_i|], \quad (18)$$

where $g_i = t_i^{sw} - t_i^{hw}$ and $ga_i = g_i / \max(A_i, \epsilon)$. These features are max-normalized and used by the optional learned edge-weight module.

Edge-weight mechanism. The code supports both static and learned edge weighting. The static component can be derived from communication magnitude, cosine similarity, or hybrid variants, while the learned component is either a per-edge parameter or an MLP applied to edge features. In the common **Diff-GNN-Order** configuration, the base edge mode is `paper2_cosine` and the

learned edge-weight mechanism is `mlp`, with edge-weight scales clipped to $[0.5, 1.5]$.

4.2 Order-Aware Graph Encoder

Let $h_i^{(0)}$ be the normalized input feature for node i . A shared GCN-based encoder with L layers computes node embeddings

$$h_i^{(\ell+1)} = \text{GNN}^{(\ell)}\left(h_i^{(\ell)}, \{h_j^{(\ell)} : (j, i) \in \mathcal{E}\}, e_{ji}\right), \quad \ell = 0, \dots, L-1. \quad (19)$$

The default order-aware model in the repository uses a three-layer encoder with hidden dimension 256 and dropout 0.2 on the large SqueezeNet benchmark; the smaller 11-node configuration uses the same structure with hidden dimension 128 for faster sweeps.

The final embedding is passed to three heads:

$$\tilde{p}_i = \sigma\left(\text{Head}_{\text{place}}(h_i^{(L)})\right), \quad (20)$$

$$\phi_i^{sw} = \text{Head}_{\text{sw}}(h_i^{(L)}), \quad \phi_i^{hw} = \text{Head}_{\text{hw}}(h_i^{(L)}). \quad (21)$$

The placement head predicts the relaxed hardware assignment. The software-order head predicts the software execution priority. The hardware-order head is implemented in the code, but the stable configuration used in this paper keeps `use_hw_ordering=False`; consequently, software ordering is the active learned ordering signal.

4.3 Soft Ordering and Differentiable Scheduler

Soft software permutation. The software-priority vector is converted into a soft permutation through Sinkhorn normalization:

$$\mathbf{P}^{sw} = \text{Sinkhorn}\left(\frac{\phi^{sw}}{\tau_o}\right), \quad (22)$$

where τ_o is the ordering temperature. We use 12 Sinkhorn iterations in the repository’s fast and default order-aware settings, and Gumbel noise is disabled in the stable benchmark configuration.

Pairwise-before relation. The pairwise ordering matrix is approximated from expected ranks:

$$\mathbf{B}_{ji}^{sw} = \sigma\left(\frac{\tilde{r}_i - \tilde{r}_j}{\tau_r}\right), \quad \tilde{r}_i = \sum_{a=0}^{|\mathcal{V}|-1} a \mathbf{P}_{ai}^{sw}, \quad (23)$$

where τ_r is the pairwise-temperature parameter. The code uses `pairwise_mode=rank_sigmoid` with $\tau_r = 0.35$.

Placement-gated software serialization. Ordering matters only when both tasks are assigned to software. Therefore, the effective software-order matrix is

$$\mathbf{R}_{ji}^{sw} = \mathbf{B}_{ji}^{sw}(1 - \tilde{p}_j)(1 - \tilde{p}_i). \quad (24)$$

The relaxed DAG-readiness term follows Equation (11). The software-readiness term uses the learned order:

$$s_i^{sw} = \text{SMax}_{\beta}\left(\{\tilde{f}_j + \alpha \log(\mathbf{R}_{ji}^{sw} + \epsilon) \mid j \neq i\}\right), \quad (25)$$

where α is the resource-order scaling factor.

The relaxed start and finish times are then computed as in Equations (13) and (14). This yields the order-aware relaxed makespan $\widetilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw})$.

4.4 Training Objective

The training loss combines the relaxed makespan surrogate with feasibility and regularization terms:

$$\mathcal{L}_{\text{ord}} = \widetilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw}) + \lambda_a \mathcal{L}_{\text{area}}(\tilde{\mathbf{p}}) + \lambda_c \widetilde{C}(\tilde{\mathbf{p}}) + \lambda_{\text{perm}} \Omega(\mathbf{P}^{sw}), \quad (26)$$

where $\widetilde{C}(\tilde{\mathbf{p}})$ is the relaxed partition-cost term and $\Omega(\mathbf{P}^{sw})$ contains optional doubly stochastic and entropy regularizers. In the benchmark configuration used here, the partition-cost coefficient is set to 0.01 on the small graph and 0.0833333333 in the default SqueezeNet configuration, while permutation regularizers are disabled.

4.5 Discrete Decode, Repair, and Reporting

At inference time, the relaxed placement is discretized, repaired, and post-processed:

1. threshold the relaxed placement scores and build one or more decode candidates,
2. repair any area-constraint violation by moving selected tasks back to software,
3. optionally refill unused hardware area according to the decode score,
4. apply hybrid LSSP-based local search.

The hybrid post-process used in our experiments employs LSSP evaluation, area fill, node swapping, two DLS refinement steps, and a maximum of 120 local-search iterations.

We compute three schedule metrics for the repaired partition:

$$M_{\text{DAG}}, \quad M_{\text{static}}, \quad M_{\text{learned}}. \quad (27)$$

M_{DAG} is the DAG makespan without the learned software-priority signal. M_{static} is the LSSP makespan

under static priorities computed from the repaired partition. M_{learned} is the LSSP makespan under the learned software-priority scores and is only defined for **Diff-GNN-Order**.

The reported value is

$$M_{\text{report}} = \begin{cases} M_{\text{static}}, & \text{for random, greedy, GCPs, PSO, DBPSO, CLPSO, CCPSO, ESA, SHADE, JADE, GL25, and} \\ \min(M_{\text{static}}, M_{\text{learned}}), & \text{for Diff-GNN-Order.} \end{cases} \quad (28)$$

We still list M_{DAG} in the results tables because it provides a schedule baseline shared by all methods.

4.6 Relation to the Placement-Only Ablation

The placement-only **Diff-GNN** ablation uses the same graph encoder and relaxed placement mechanism but removes the ordering heads and the order-aware scheduling terms. Its reported result is therefore based on M_{static} only. This ablation isolates the value of learned ordering and is the primary within-family comparison in the experimental section.

5 Experimental Settings

5.1 Datasets

The real benchmark suite is stored under `inputs/task_graph_topology/soda-benchmark-graphs`. It contains seven TOSA task graphs: three PyTorch-derived graphs (`mobile_net_tosa.dot`, `rez_net_tosa.dot`, and `squeeze_net_tosa.dot`) and four TFLite-derived graphs (`anomaly_detection_tosa.dot`, `image_classification_tosa.dot`, `keyword_spotting_tosa.dot`, and `visual_wake_words_tosa.dot`). Across these seven real DAGs, the benchmark range spans from 68 nodes and 63 edges up to 830 nodes and 1066 edges. SqueezeNet-TOSA remains the primary neural-network benchmark discussed throughout the paper, with 179 nodes and 234 directed edges. In addition to the SODA suite, we keep the custom sanity-check graph `inputs/task_graph_topology/custom/paper_fig3_11node.dot`, which contains 11 nodes and 12 edges and is used for controlled visualizations and ablations.

To study scalability beyond the real benchmarks, we generate SqueezeNet-like synthetic DAGs that preserve the edge-to-node ratio of the SqueezeNet template. In the current setup, we materialize synthetic graphs with $N \in \{11, 50, 1000, 10000\}$ nodes, which produce 14, 65, 1307, and 13073 edges, respectively. Table 1 summarizes the seven real SODA graphs together with the four synthetic SqueezeNet-like graphs used for scaling. In the makespan comparison, all non-order-aware baselines

are reported by their discrete static-priority makespan M_{static} , whereas **Diff-GNN-Order** reports both M_{learned} and

$$M_{\text{report}} = \min(M_{\text{static}}, M_{\text{learned}}). \quad (29)$$

Concrete task-graph instances are created by calling `TaskGraph.load_graph_from_pydot` on the real topologies or by loading the generated synthetic pickles under `inputs/task_graph_complete/`. The resulting instances are cached under `inputs/task_graph_complete/` using filenames of the form

`taskgraph-squeeze_net_tosa-instance-config-<config>.pkl`.

The cached SqueezeNet sweep in the repository spans area ratios $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, hardware scale factors $k \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$, fixed variance $l = 0.5$, fixed communication scale $\mu = 1.0$, and multiple random seeds.

5.2 Cost Model

We use the cost-assignment mechanism implemented in `meta_heuristic/task_graph.py`. For every node i , the software execution cost is sampled as

$$t_i^{sw} \sim \mathcal{U}(1, 100). \quad (30)$$

The hardware execution cost is sampled from

$$t_i^{hw} = \max(0.1, \mathcal{N}(k \cdot t_i^{sw}, (l \cdot k \cdot t_i^{sw})^2)), \quad (31)$$

where k is the hardware scale factor and l is the hardware-cost variance. Hardware area is sampled as

$$A_i \sim \mathcal{U}(1, A_{\text{max}}), \quad (32)$$

and the communication cost on edge (j, i) is sampled as

$$C_{ji} \sim \mathcal{U}(0, 2\mu s_{\text{max}}), \quad s_{\text{max}} = \max_{v \in \mathcal{V}} t_v^{sw}, \quad (33)$$

where μ is the communication scale factor. A partition is feasible only if

$$\sum_{i \in \mathcal{V}} p_i A_i \leq \alpha \sum_{i \in \mathcal{V}} A_i. \quad (34)$$

5.3 Proposed-Method Settings

Unless otherwise noted, the reported **Diff-GNN-Order** experiments use the order-aware configuration implemented in `diff_gnn_ordering.py`: a three-layer GCN encoder with hidden dimension 256, dropout 0.2, `feature_profile=default_plus_paper`, `edge_weight_mode=paper2_cosine`, `edge_weight_learner=mlp`, 12 Sinkhorn iterations, two order-refinement steps, `pairwise_mode=rank_sigmoid`,

Table 1: Topology statistics for the seven real SODA benchmark graphs and the synthetic SqueezeNet-like suite.

Family	Graph	$ \mathcal{V} $	$ \mathcal{E} $
PyTorch-TOSA	<code>mobile_net_tosa</code>	830	1066
PyTorch-TOSA	<code>rez_net_tosa</code>	362	457
PyTorch-TOSA	<code>squeeze_net_tosa</code>	179	234
TFLite-TOSA	<code>anomaly_detection_tosa</code>	68	63
TFLite-TOSA	<code>image_classification_tosa</code>	125	144
TFLite-TOSA	<code>keyword_spotting_tosa</code>	123	139
TFLite-TOSA	<code>visual_wake_words_tosa</code>	211	227
Synthetic	<code>squeezenet_like_11</code>	11	14
Synthetic	<code>squeezenet_like_50</code>	50	65
Synthetic	<code>squeezenet_like_1000</code>	1000	1307
Synthetic	<code>squeezenet_like_10000</code>	10000	13073

pairwise temperature 0.35, no Gumbel noise, and `use_hw_ordering=False`. Training and final selection both use the queue-style differentiable metric, while the final discrete solution is refined with hybrid LSSP post-processing. On the small 11-node graph, we use the corresponding fast profile with hidden dimension 128 and 500 training iterations. The placement-only Diff-GNN ablation uses the same encoder and post-processing pipeline but removes the ordering heads.

5.4 Competing Methods

The comparison set is taken directly from `gnn_main.py`. It spans simple heuristics, population-based metaheuristics, and prior graph-learning baselines: random assignment and a gain-per-area greedy heuristic as lightweight references; a CVXPY-backed MILP formulation as an exact small-graph reference; PSO and DBPSO as canonical swarm optimizers [5, 6]; CLPSO and CCPSO as more advanced particle-swarm variants [7, 8]; ESA, SHADE, JADE, and GL25 as representative simulated-annealing, differential-evolution, and genetic-algorithm baselines [12, 9, 10, 11]; and GCPS as a prior GNN-based HW/SW partitioning approach [4]. We also include the placement-only Diff-GNN model as an ablation to isolate the contribution of the proposed ordering head.

5.5 Evaluation Protocol

For every repaired partition, we compute three makespan metrics:

$$M_{\text{DAG}}, \quad M_{\text{static}}, \quad M_{\text{learned}}. \quad (35)$$

M_{DAG} is the baseline DAG makespan. M_{static} is the makespan obtained by LSSP with static priorities derived from the partition. M_{learned} is the LSSP makespan obtained by using the learned software-priority scores; it is only available for `Diff-GNN-Order`.

The reported metric is

$$M_{\text{report}} = \begin{cases} M_{\text{static}}, & \text{for all non-order-aware methods} \\ \min(M_{\text{static}}, M_{\text{learned}}), & \text{for Diff-GNN-Order.} \end{cases} \quad (36)$$

We still list M_{DAG} in every table as a baseline for comparison.

5.6 Synthetic Scaling Benchmark

To study scalability beyond the real TOSA graphs, we add a SqueezeNet-like synthetic generator to the codebase. The generator uses `squeeze_net_tosa.dot` as the structural template, extracts its topological-layer profile and inter-layer edge pattern, and then produces a larger DAG with the same edge-to-node ratio:

$$\frac{|\mathcal{E}_{\text{syn}}|}{|\mathcal{V}_{\text{syn}}|} \approx \frac{234}{179}. \quad (37)$$

For a requested node count N , the target edge count is

$$E_{\text{syn}} = \text{round}\left(\frac{234}{179}N\right). \quad (38)$$

In this paper, we use $N \in \{11, 50, 1000, 10000\}$. After generating the synthetic topology, node and edge costs are sampled with the same (k, l, μ, α) cost model as on the real graphs.

6 Experimental Results

This section reports an initial single-run pilot used to populate the current draft. The full multi-seed campaign is still in progress, so the numbers below should be interpreted as preliminary sanity-check results rather than the final averages of the paper. For these pilot runs, we keep the evaluation protocol fixed, but reduce the per-method runtime budgets so that all baselines can be executed once on the three requested graph families. Following Sec. 5, every non-order-aware method is reported

Table 2: Baseline settings used in the codebase.

Method	Full form / implementation basis	Settings used in the repository
Random	Uniform random HW/SW assignment	500 samples, Bernoulli probability $p = 0.5$
Greedy	Gain-per-area hardware selection heuristic	Sort by $(t_i^{sw} - t_i^{hw})/A_i$ and fill hardware greedily
MILP	Mixed-Integer Linear Programming via CVXPY	hybrid solve mode, adjacent SW constraints, reduced SW constraints enabled, round threshold 0.5, preferred solvers {HIGHS, GUROBI, SCIP, GLPK_MI, CBC, ECOS_BB}, time limit 30 s, MIP gap 0.15, node limit 20,000, accept non-optimal solutions
PSO	Particle Swarm Optimization [5]	$c_1 = 0.575$, $c_2 = 0.1$, $w = 1.05$, 500 particles, 10 iterations
DBPSO	Discrete Binary Particle Swarm Optimization [6]	$c_1 = 0.575$, $c_2 = 0.1$, $w = 1.05$, neighborhood $k = 4$, Minkowski $p = 2$, 500 particles, 10 iterations
CLPSO	Comprehensive Learning PSO [7]	learning coefficient $c = 1$, 500 individuals, 200 evaluations/iterations budget
CCPSO	Cooperative Coevolutionary PSO [8]	$c = 1$, 500 individuals, 200 evaluations/iterations budget, group sizes {5, 10, 20}
ESA	Enhanced Simulated Annealing [12]	1,000 function evaluations
SHADE	Success-History Adaptive Differential Evolution [9]	2,000 function evaluations, 500 individuals
JADE	Adaptive Differential Evolution with Archive [10]	2,000 function evaluations, 500 individuals
GL25	Global-local real-coded GA [11]	2,000 function evaluations, population size 500
GCPS	Graph-convolution partitioning baseline [4]	learning rate 10^{-3} , dropout 0.2, hidden dimensions 10/5, pretrain 100, max epochs 800, schedule skip 5, $\sigma = 0.3$, LSSP evaluation
Diff-GNN	Placement-only differentiable GNN	Same encoder and post-processing as Diff-GNN-Order , but no ordering heads

by its static-priority makespan, while **Diff-GNN-Order** is reported by

$$M_{\text{report}} = \min(M_{\text{static}}, M_{\text{learned}}). \quad (39)$$

We do not include the MILP baseline in the SqueezeNet-TOSA and synthetic pilot comparisons because those graphs are intended to probe the regime where exact optimization is no longer practical.

6.1 Initial Results on Three Graph Families

We prepare an initial comparison on three task graphs: the 11-node paper graph, one SqueezeNet-TOSA instance, and one 50-node synthetic SqueezeNet-like DAG. Table 3 reports the pilot *reported* makespan for every competing method. Table 4 then expands the three metrics used by **Diff-GNN-Order**.

The small 11-node graph already shows the expected value of ordering-aware decoding. The placement-only **Diff-GNN** run reaches $M_{\text{static}} = 31$, while **Diff-GNN-Order** reduces the learned-priority makespan

to 29, matching the best metaheuristic pilot results (DBPSO, SHADE, JADE, and GL25). On this graph, the learned ordering head is therefore directly responsible for the two-cycle improvement from 31 to 29.

The larger pilot results are more mixed. On the single SqueezeNet-TOSA run, Greedy and PSO both obtain the lowest pilot static-priority makespan of 8672, while **Diff-GNN-Order** is close at 8704. The learned ordering head does not improve this particular run because $M_{\text{learned}} = 8877 > M_{\text{static}} = 8704$, so the reported result falls back to the static schedule. On the 50-node synthetic graph, the placement-only **Diff-GNN** pilot gives the best reported makespan of 1490, clearly ahead of the non-learning baselines, whereas **Diff-GNN-Order** reaches 2075. This indicates that the current short-budget order model is not yet tuned for the synthetic regime, and that the final large-scale study should be based on the full training budgets and multi-seed averages rather than the current pilot alone.

Table 3: Initial single-run pilot results on three graph families. All entries are reported makespan values. For every baseline, the entry equals M_{static} ; for **Diff-GNN-Order**, the entry equals $M_{\text{report}} = \min(M_{\text{static}}, M_{\text{learned}})$.

Graph	$ \mathcal{V} $	$ \mathcal{E} $	M_{DAG}	Random	Greedy	GCPs	PSO	DBPSO	CLPSO	CCPSO	ESA	SHADE	JADE	GL25	Diff-GNN	Diff-GNN-Order
Small-11	11	12	32	32	32	32	83	29	30	32	30	29	29	29	31	29
SqueezeNet-TOSA	179	234	8527	11910	8672	10038	8672	11261	10735	12874	11765	12637	12299	11765	9189	8704
Synthetic-50	50	65	703	3280	2257	2067	2257	2918	2831	3159	2815	2953	3097	2815	1490	2075

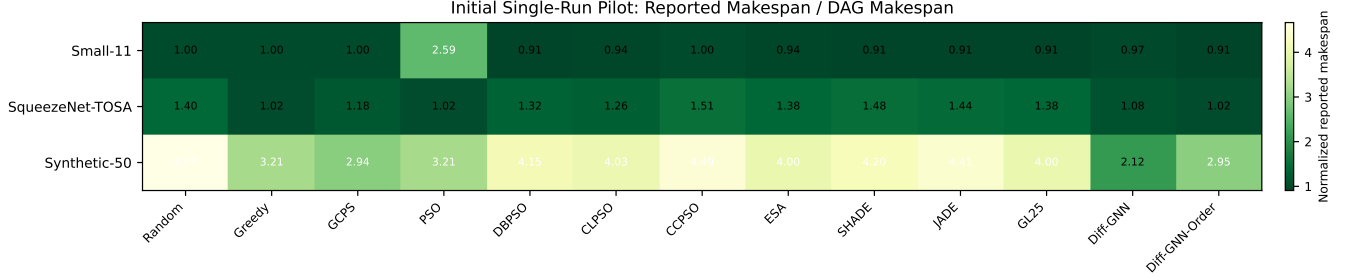


Figure 1: Initial pilot heatmap of normalized reported makespan, i.e., $M_{\text{report}}/M_{\text{DAG}}$. Lower is better. The figure makes cross-graph behavior easier to compare because the three graphs have very different absolute schedule lengths.

Table 4: Detailed metrics for **Diff-GNN-Order** in the initial pilot.

Graph	M_{DAG}	M_{static}	M_{learned}	M_{report}
Small-11	32	31	29	29
SqueezeNet-TOSA	8527	8704	8877	8704
Synthetic-50	703	2075	2284	2075

Table 6: Edge-weight ablation template for **Diff-GNN-Order** on the Small-11 graph.

Setting	M_{DAG}	M_{static}	M_{learned}	M_{report}
Learned edge weights (MLP)	32	31	29	29
Fixed edge weights	32	TBD	TBD	TBD

Table 5: Main ablation table for **Diff-GNN-Order** on the Small-11 graph.

Variant	M_{DAG}	M_{static}	M_{learned}	M_{report}
Diff-GNN without ordering head	32	31	N/A	31
Diff-GNN-Order with ordering head	32	31	29	29
Diff-GNN-Order without post-processing	32	TBD	TBD	TBD
Diff-GNN-Order without refinement	32	TBD	TBD	TBD

6.2 Ablation Studies

Table 5 summarizes the currently available ablation values. The first two rows are filled from the completed small-graph run, which already confirms the benefit of the ordering head. The remaining rows are left as explicit placeholders in this draft and will be populated after the dedicated ablation rerun finishes.

The currently completed ablation rows already show that the ordering head matters: on the Small-11 graph, adding ordering reduces the reported makespan from 31 to 29 even when the underlying placement backbone is held fixed. The remaining ablations are kept in the manuscript as ready-to-fill placeholders so that the final version can directly incorporate the dedicated rerun without restructuring the section.

7 Conclusion

This paper positions **Diff-GNN-Order** as the main method for scheduling-aware HW/SW partitioning and uses the placement-only **Diff-GNN** model strictly as an ablation. The manuscript now aligns the notation, algorithm description, and evaluation protocol with the ICCAD-style presentation: every method is compared against the DAG baseline, non-order-aware methods are reported by static-priority makespan, and **Diff-GNN-Order** is reported by the better of its static and learned-priority schedules.

The initial pilot experiments already show the main effect that motivates the method. On the Small-11 graph, the ordering head improves the reported makespan from 31 to 29, matching the best pilot metaheuristic results. On the larger SqueezeNet-TOSA and Synthetic-50 pilots, the results are more mixed: **Diff-GNN-Order** remains competitive, but the learned ordering does not yet consistently outperform the strongest static baselines under the reduced-budget draft runs. The final version of the paper will therefore complete the full multi-seed evaluation, populate the remaining ablation rows for post-processing, refinement, and fixed edge weights, and report the definitive large-scale synthetic results at 1,000 and 10,000 nodes.

A Notation and Pseudocode

A.1 Main Notation

Table 7: Main notation used in the paper.

Symbol	Meaning
$\mathcal{G}(\mathcal{V}, \mathcal{E})$	Task graph
(j, i)	Dependency edge from task j to task i
t_i^{sw}, t_i^{hw}	Software and hardware execution costs
$A_i, A_{\max}$	Node area cost and total hardware-area budget
C_{ji}	Communication cost on edge (j, i)
$\mathbf{p}, \tilde{\mathbf{p}}$	Discrete and relaxed placements
$\pi^{sw}, \tilde{\pi}^{sw}$	Discrete and relaxed software orders
$\mathbf{s}, \mathbf{f}$	Discrete start and finish times
$\tilde{\mathbf{s}}, \tilde{\mathbf{f}}$	Relaxed start and finish times
M_{DAG}	DAG makespan baseline
M_{static}	Static-priority makespan
M_{learned}	Learned-priority makespan
$\widetilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw})$	Relaxed makespan surrogate
$\text{SMax}_{\beta}(\cdot)$	Smooth maximum operator

A.2 Order-Aware Training and Decode

Algorithm 1: Training of Diff-GNN-Order.

1. Construct node and edge features for the input task graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$.
2. Run the shared encoder and predict the relaxed placement vector $\tilde{\mathbf{p}}$ and software-priority vector $\tilde{\pi}^{sw}$.
3. Build the soft software permutation $\mathbf{P}^{sw}$ with Sinkhorn normalization and convert it to the pairwise-before matrix $\mathbf{B}^{sw}$.
4. Form the effective software-order matrix $\mathbf{R}^{sw}$ using Equation (24).
5. Evaluate the relaxed start and finish times with the differentiable scheduler and compute $\widetilde{M}(\tilde{\mathbf{p}}, \tilde{\pi}^{sw})$.
6. Add the area penalty and auxiliary regularizers to obtain $\mathcal{L}_{\text{ord}}$.
7. Update the model parameters by backpropagation.

Algorithm 2: Discrete inference and reporting.

1. Decode one or more candidate binary partitions from the relaxed placement scores.
2. Repair area violations and optionally refill unused hardware area.
3. Apply hybrid LSSP local search to the repaired partition.

4. Compute the DAG baseline M_{DAG} and the static-priority makespan M_{static} .
5. If learned software-priority scores are available, compute M_{learned} and report $M_{\text{report}} = \min(M_{\text{static}}, M_{\text{learned}})$; otherwise report $M_{\text{report}} = M_{\text{static}}$.

A.3 Synthetic Scaling Generator

The new synthetic benchmark generator is implemented in `figures/synthetic.py` and exposed through `tools/generate_squeezenet_like_synthetic_graph.py`. It uses the SqueezeNet-TOSA topology as a template, preserves its topological-layer profile and edge-to-node ratio, and then resamples node and edge costs using the same (k, l, μ, α) model as the real graphs.

References

References

- [1] P. Arató, Z. A. Mann, and A. Orbán, “Algorithmic aspects of hardware/software partitioning,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 10, no. 1, pp. 136–156, 2005.
- [2] N. Hou, X. Yan, and F. He, “A survey on partitioning models, solution algorithms and algorithm parallelization for hardware/software co-design,” *Design Automation for Embedded Systems*, vol. 23, no. 1–2, pp. 57–77, 2019.
- [3] J. Teich, “Hardware/software codesign: The past, the present, and predicting the future,” *Proceedings of the IEEE*, vol. 100, pp. 1411–1430, 2012.
- [4] X. Zheng, S. Liang, and X. Xiong, “A hardware/software partitioning method based on graph convolution network,” *Design Automation for Embedded Systems*, vol. 25, no. 4, pp. 325–351, 2021.
- [5] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proceedings of the IEEE International Conference on Neural Networks*, 1995, pp. 1942–1948.
- [6] J. Kennedy and R. C. Eberhart, “A discrete binary version of the particle swarm algorithm,” in *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics*, 1997, pp. 4104–4108.
- [7] J. J. Liang, A. K. Qin, P. N. Suganthan, and S. Baskar, “Comprehensive learning particle swarm optimizer for global optimization of multimodal functions,” *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 3, pp. 281–295, 2006.

- [8] X. Li and X. Yao, “Cooperatively coevolving particle swarms for large scale optimization,” *IEEE Transactions on Evolutionary Computation*, vol. 16, no. 2, pp. 210–224, 2011.
- [9] R. Tanabe and A. Fukunaga, “Success-history based parameter adaptation for differential evolution,” in *Proceedings of the IEEE Congress on Evolutionary Computation*, 2013, pp. 71–78.
- [10] J. Zhang and A. C. Sanderson, “JADE: Adaptive differential evolution with optional external archive,” *IEEE Transactions on Evolutionary Computation*, vol. 13, no. 5, pp. 945–958, 2009.
- [11] C. García-Martínez, M. Lozano, F. Herrera, D. Molina, and A. M. Sánchez, “Global and local real-coded genetic algorithms based on parent-centric crossover operators,” *European Journal of Operational Research*, vol. 185, no. 3, pp. 1088–1113, 2008.
- [12] P. Siarry, G. Berthiau, F. Durdin, and J. Haussy, “Enhanced simulated annealing for globally minimizing functions of many continuous variables,” *ACM Transactions on Mathematical Software*, vol. 23, no. 2, pp. 209–228, 1997.
- [13] G. Mena, D. Belanger, S. Linderman, and J. Snoek, “Learning latent permutations with Gumbel-Sinkhorn networks,” in *Proceedings of ICLR*, 2018.